Lab 1: Introduction to OpenMP: Enabling OpenMP and Executing Programs in Parallel

Objectives: After completing this laboratory, students will be able to:

1. Compile and execute OpenMP programs using GCC.
2. Understand the OpenMP Fork–Join execution model.
3. Create parallel regions using the `parallel` directive.
4. Obtain thread information using OpenMP runtime library functions.
5. Parallelize iterative loops using the `for` work-sharing construct.
6. Compare serial and parallel execution of loops.
7. Measure execution time using OpenMP runtime functions.

OpenMP Overview

OpenMP (Open Multi-Processing) is an Application Programming Interface (API) for shared-memory parallel programming in C, C++, and Fortran. It is jointly developed and maintained by the OpenMP Architecture Review Board (ARB).OpenMP enables programmers to develop parallel applications using three main components:

- Compiler directives (Pragmas): Used to specify parallel regions and work-sharing constructs.
- Runtime library routines: Functions for controlling threads, obtaining thread information, and measuring execution time.
- Environment variables: Variables that configure the execution environment, such as the number of threads and scheduling policies.

OpenMP follows a Fork–Join execution model, in which a single master thread creates multiple worker threads to execute parallel regions concurrently. Once the parallel work is completed, the worker threads synchronize and terminate, allowing the master thread to continue sequential execution.

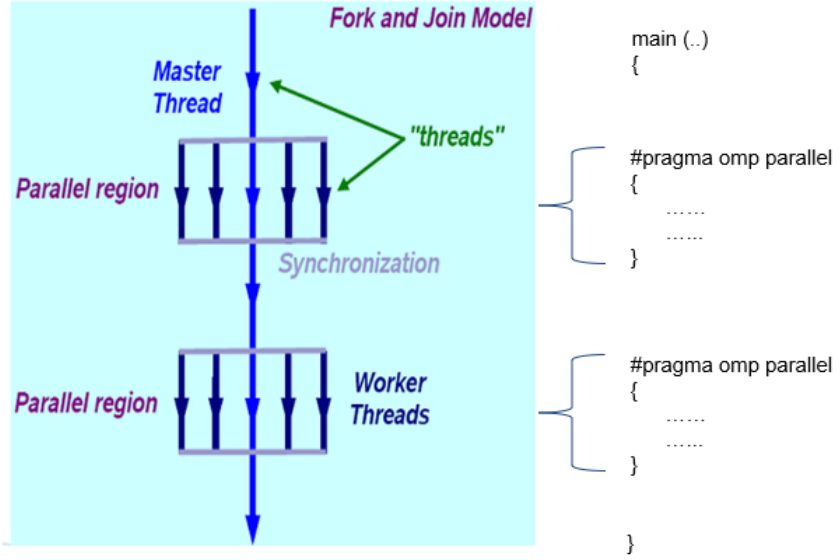

Fig. 1: Fork-Join Execution Model of OpenMP

1. Header File

To use OpenMP constructs and runtime library functions, the OpenMP header file must be included in every program.

Syntax: #include <omp.h>
Explanation: The omp.h header file contains the declarations of all OpenMP runtime library functions, such as:

omp_get_thread_num()
omp_get_num_threads()
omp_set_num_threads()
omp_get_wtime()

Without including this header file, the compiler cannot recognize these functions.

2. Compiling an OpenMP Program

OpenMP directives are recognized only when the compiler is instructed to enable OpenMP support.

GCC Compilation Syntax
gcc -fopenmp filename.c -o filename
Example
gcc -fopenmp hello.c -o hello

Execute the program using

./hello

The compiler option -fopenmp enables OpenMP support and automatically links the required OpenMP runtime library.

3. OpenMP Compiler Directives

OpenMP uses compiler directives (also called pragmas) to specify parallel regions and work-sharing constructs. These directives are ignored by compilers that do not support OpenMP, allowing the same program to execute sequentially.

The general syntax of an OpenMP directive is

#pragma omp directive-name [clause(s)]

where: #pragma tells the compiler that a special instruction follows.
omp indicates that the directive belongs to OpenMP.
directive-name specifies the OpenMP construct.
clause(s) are optional parameters that modify the behavior of the directive.

4. The parallel Directive

The parallel directive creates a team of threads to execute the enclosed block of code concurrently. When a thread encounters a parallel directive, it becomes the master thread and forks into multiple worker threads. Each thread executes all the statements within the parallel region independently.

Syntax

```
#pragma omp parallel
{
   // Parallel code
}
```

Key Points

- Creates a team of threads.
- Every thread every statement inside the parallel region.
- Thread creation executes occurs only at the beginning of the parallel region.
- At the end of the parallel region, all threads synchronize and terminate.
- Execution continues with the master thread after the parallel region.

Example

```
#pragma omp parallel
{
   printf("Hello OpenMP\n");
}
```

If four threads are created, the message is printed four times.

5. OpenMP Runtime Library Functions

OpenMP provides runtime library routines for thread management and performance measurement.

5.1 omp_get_thread_num()

This function returns the identification number (Thread ID) of the calling thread.

Syntax

```
int omp_get_thread_num();
```

Purpose

Identifies the currently executing thread.
Thread IDs begin from 0.
The master thread always has ID 0.
Example Output

```
Thread 0
Thread 2
Thread 1
Thread 3
```

Since threads execute concurrently, the output order is generally non-deterministic.

5.2 omp_get_num_threads()

Returns the total number of threads in the current team.

Syntax
int omp_get_num_threads();
Purpose
Determines the number of active threads inside a parallel region.
Returns 1 when called outside a parallel region.
Example
printf("%d", omp_get_num_threads());

Output : 4

5.3 omp_set_num_threads()

Specifies the number of threads to be created for subsequent parallel regions.

Syntax
void omp_set_num_threads(int num_threads);
Example
omp_set_num_threads(8);

The next parallel region will attempt to create eight threads.

Notes
Must be called before entering the parallel region.
Overrides the default number of threads unless restricted by the system or environment variables.

5.4 omp_get_wtime()

Returns the elapsed wall-clock time in seconds.

Syntax
double omp_get_wtime();
Purpose
Measures program execution time.
Useful for comparing serial and parallel implementations.
Returns time in seconds with high resolution.
Example

```
double start, end;

start = omp_get_wtime();

/* Parallel code */

end = omp_get_wtime();

printf("Execution Time = %f", end - start);
```

6. Parallel Loop Using for

The for directive distributes the iterations of a loop among multiple threads. Each iteration is executed exactly once by one of the available threads.

Syntax

```
#pragma omp parallel for
for(initialization; condition; increment)
{
   statements;
}
```

Working

Suppose the loop contains 100 iterations and four threads are available.

Each thread may execute approximately 25 iterations.

Example distribution:

Thread 0 → Iterations 0–24

Thread 1 → Iterations 25–49

Thread 2 → Iterations 50–74

Thread 3 → Iterations 75–99

The exact assignment depends on the scheduling policy.

Advantages
Automatic loop parallelization.
Balanced workload distribution.
Easy to convert serial loops into parallel loops.

7. Difference Between parallel and parallel for

parallel	**parallel for**
Creates multiple threads.	Creates multiple threads and automatically distributes loop iterations.
Every thread executes all statements inside the block.	Each loop iteration is executed by only one thread.
Programmer divides work manually.	Work is divided automatically by OpenMP.
Suitable for general parallel regions.	Suitable for data-parallel loops.

8. Best Practices

- Include omp.h in every OpenMP program.
- Compile all OpenMP programs using the -fopenmp compiler option.
- Use parallel to create parallel regions.

- Use parallel for to parallelize loops.
- Use omp_get_thread_num() for debugging and understanding thread execution.
- Use omp_get_num_threads() to verify the number of threads created.
- Measure execution time using omp_get_wtime() when comparing serial and parallel implementations.

Compilation and Execution Command:

gcc -fopenmp filename.c -o filename

Example: gcc -fopenmp hello.c -o hello

./hello

Sample Program: Implement and execute a simple OpenMP program that creates multiple threads and prints a "Hello World" message from each thread.

```
#include<stdio.h>
#include<omp.h>

int main()
{

#pragma omp parallel
{
   printf("Hello OpenMP!\n");
}

return 0;

}
```

Sample Output:

```
Hello OpenMP!
Hello OpenMP!
Hello OpenMP!
Hello OpenMP!
```

Sample Program:Implement an OpenMP program that displays the unique identification number (Thread ID) of each thread executing the parallel region.

```
#include<stdio.h>
#include<omp.h>

int main()
{

#pragma omp parallel
```

```
{
    int id;

    id = omp_get_thread_num();

    printf("Hello from Thread %d\n",id);
}

return 0;

}
```

Sample Output:

```
Hello from Thread 0
Hello from Thread 2
Hello from Thread 3
Hello from Thread 1
```

Lab Exercise:

1. Implement an OpenMP program to determine and display the total number of threads participating in a parallel region using the omp_get_num_threads() runtime function.
2. Implement an OpenMP program that creates a user-specified number of threads using the omp_set_num_threads() runtime function and displays the Thread ID of each thread.
3. Implement an OpenMP program to initialize all elements of a 5 × 5 matrix with consecutive integers inside a parallel region using the parallel directive. Display the initialized matrix and print the Thread ID responsible for initializing each row.
4. Implement an OpenMP program to perform the addition of two one-dimensional arrays of size N using the parallel for directive. Display the resulting array and identify the Thread ID that computes each array element.
5. Implement an OpenMP program to perform the addition of two matrices of size M×N using the parallel for directive. Display the resultant matrix and indicate the Thread ID responsible for computing each row (or element) of the result.
6. Implement an OpenMP program to compute the sum of elements of a large array in parallel using the parallel for directive. Measure and display the execution time of the parallel program using the omp_get_wtime() runtime function and compare it with the corresponding serial implementation.

Additional Exercise:

1. Implement a program in C to reverse the digits of the following integer array of size 9. Initialize the input array to the following values.
   Input array: 18, 523, 301, 1234, 2, 14, 108, 150, 1928
   Output array: 81, 325, 103, 4321, 2, 41, 801, 51, 8291

2. Implement a program in C to simulate the all the operations of a calculator. Given inputs A and B, find the output for A+B, A-B, A*B and A/B.

3. Implement a program in C to toggle the character of a given string. Example: suppose the string is “HeLLo”, then the output should be “hEllO”.
4. Implement a C program to read a word of length N and produce the pattern as shown in the example. Example: Input: PCBD Output: PCCBBBDDDD

5. Implement a C program to read two strings S1 and S2 of same length and produce the resultant string as shown below.

   S1: string S2: length Resultant String: slternigntgh

6. Implement a C program to perform Matrix times vector product operation.
7. Implement a C program to read a matrix A of size 5x5. It produces a resultant matrix B of size 5x5. It sets all the principal diagonal elements of B matrix with 0. It replaces each row elements in the B matrix in the following manner. If the element is below the principal diagonal it replaces it with the maximum value of the row in the A matrix having the same row number of B. If the element is above the principal diagonal it replaces it with the minimum value of the row in the A matrix having the same row number of B.

E⊹mple:

A				
1	2	3	4	5
5	4	3	2	4
10	3	13	14	15
11	2	11	33	44
1	12	5	4	6

B				
0	1	1	1	1
5	0	2	2	2
15	15	0	3	3
44	44	44	0	2
12	12	12	12	0

8. Implement a C program that reads a matrix of size MxN and produce an output matrix B of same size such that it replaces all the non-border elements of A with its equivalent 1’s complement and remaining elements same as matrix A. Also produce a matrix D as shown below.

Example:

A B D

1	2	3	4
6	5	8	3
2	4	10	1
9	1	2	5

1	2	3	4
6	**10**	**111**	3
2	**11**	**101**	1
9	1	2	5

1	2	3	4
6	**2**	**7**	3
2	**3**	**5**	1
9	1	2	5

9. Implement a C program that reads a character type matrix and integer type matrix B of size MxN. It produces and output string STR such that, every character of A is repeated r times (where r is the integer value in matrix B which is having the same index as that of the character taken in A).

Example:	A			B			
	C	a	P	1	2	4	3
	X	a	M	2	4	3	2

Output string STR: pCCaaaaPPPeeXXXXaaaMM

10.